



NATIONAL UNIVERSITY OF MODERN LANGUAGES

Software Construction And Development Final Project

NAME: M Abdul Rafay (FL23791)

PROGRAM: BSSE (5th Semester)

COURSE: Software Construction and Development

SUBMITTED TO: Sir Ahsan Arif

DATE: 9/21/2025

DAY: Sunday

Contents

Hospital Appointment Booking System.....	3
1. Objective.....	3
2. Introduction.....	3
3. Detailed Design.....	4
3.1 Use-Case Diagram.....	4
3.2 Object Diagram.....	5
3.3 Communication Diagram	5
3.4 Timing Diagram	6
4. Implementation.....	7
5. Test Cases.....	7

Hospital Appointment Booking System

1. Objective

The main objective of the *Hospital Appointment Booking System* is to create a digital solution that simplifies the process of booking appointments between patients and doctors. The system aims to replace manual methods with a faster and more organized platform. It helps reduce long waiting times, avoid scheduling mistakes, and keep appointment records accurate and up to date. Patients can easily book, view, or cancel appointments, while doctors and administrators can manage their schedules more effectively.

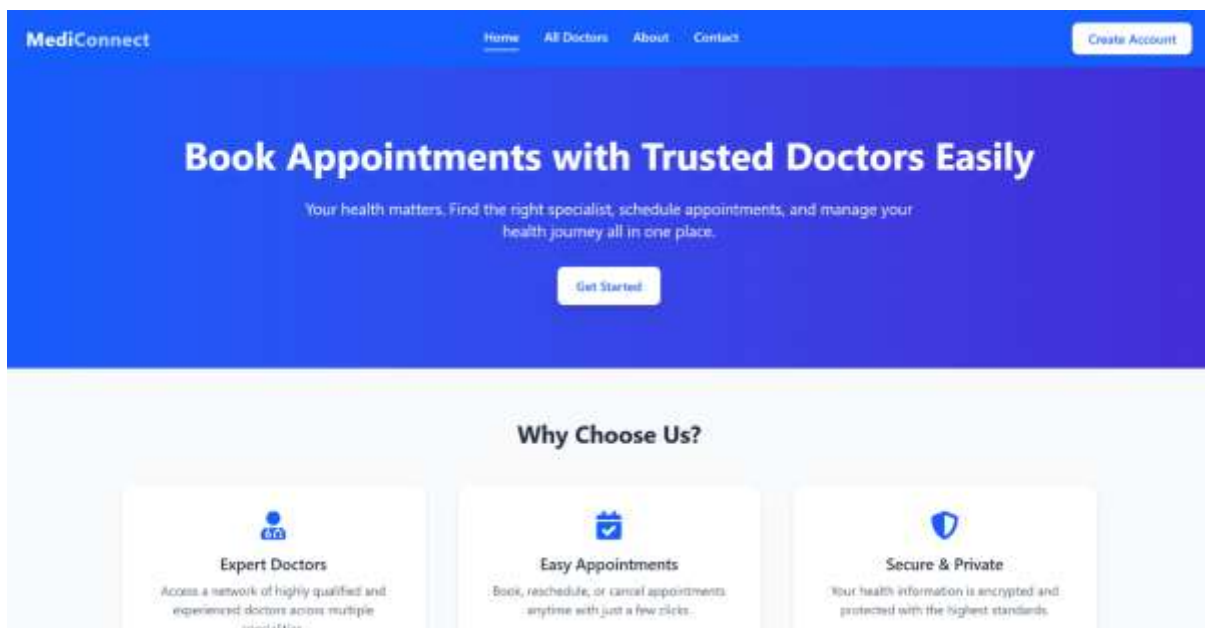
This project also aims to apply core software development practices such as requirement analysis, UML design, clean coding standards, exception handling, and unit testing. The goal is to build a reliable, user-friendly, and efficient system that supports real hospital workflows.

2. Introduction

Hospitals often face difficulties when appointments are booked manually. Long queues, busy phone lines, and misplaced records create delays and confusion for both patients and staff. As the number of patients increases, handling appointments without a proper system becomes even more difficult and time-consuming. To overcome these issues, a computerized appointment system is needed.

The *Hospital Appointment Booking System* is designed to automate and simplify the entire appointment process. Patients can register, log in, search for doctors by their specialization, and book appointments at a suitable time. Doctors can view their daily schedules and update their availability, and administrators can manage doctor profiles and time slots whenever needed.

This system improves communication, reduces workload, and ensures that appointments are handled smoothly and accurately. By using proper software engineering techniques—such as UML diagrams, structured design.



3. Detailed Design

This section explains the detailed design of the *Hospital Appointment Booking System* using UML diagrams. These diagrams help visualize the structure of the system, the relationships between objects, and the interaction flow during different operations. Each diagram provides a different perspective of the system's internal behavior and supports a clear and organized implementation.

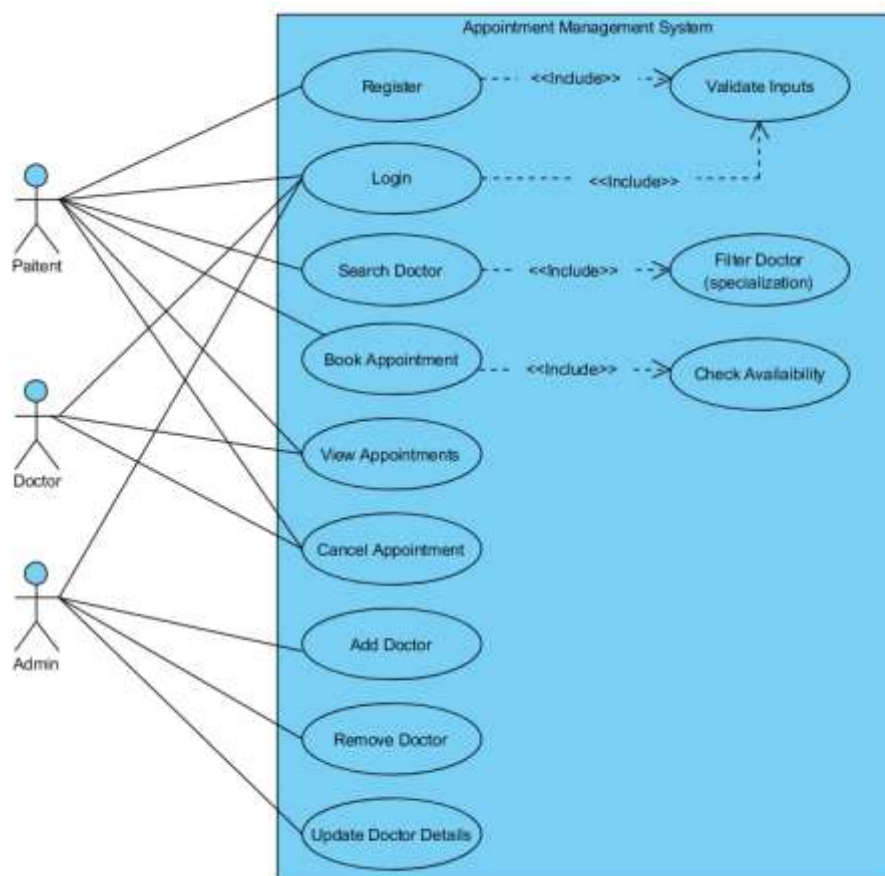
The following subsections include:

1. Use-Case Diagram
2. Object Diagram
3. Communication Diagram
4. Timing Diagram

Screenshots of each diagram are inserted below their respective descriptions.

3.1 Use-Case Diagram

The Use-Case Diagram describes the external interactions between the system and its users. It identifies the main actors of the system—**Patient**, **Doctor**, and **Admin**—and the actions they can perform.

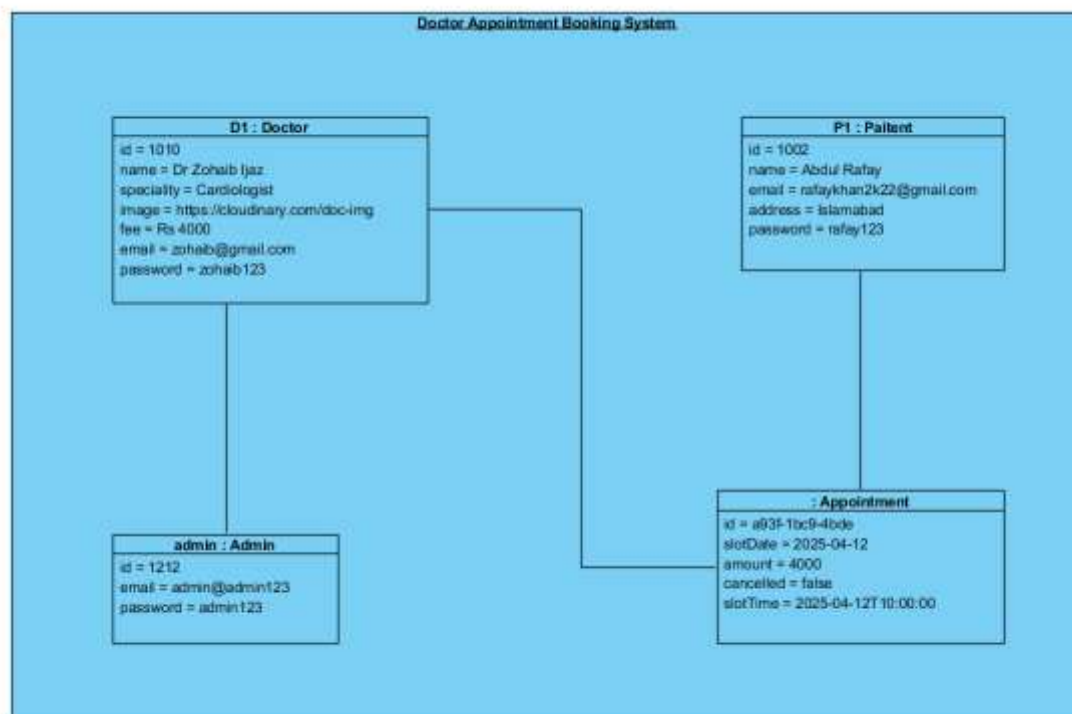


This diagram helps outline the functional requirements of the system and shows how users interact with the system to complete tasks such as booking appointments, managing doctor profiles, and viewing schedules.

3.2 Object Diagram

The Object Diagram shows a snapshot of the system at a specific moment in time. It presents actual instances of classes—such as **User (Patient)**, **Doctor**, **Admin**, and **Appointment (Oppointment)**—and displays how these objects are linked to each other.

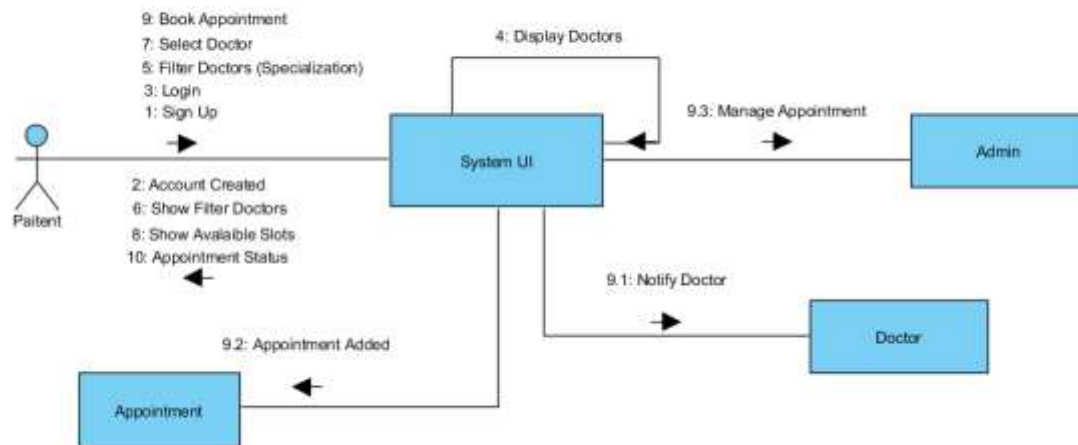
This diagram helps visualize how data is stored and connected during the appointment booking process. It also confirms that the relationships defined in the class diagram work correctly at runtime.



3.3 Communication Diagram

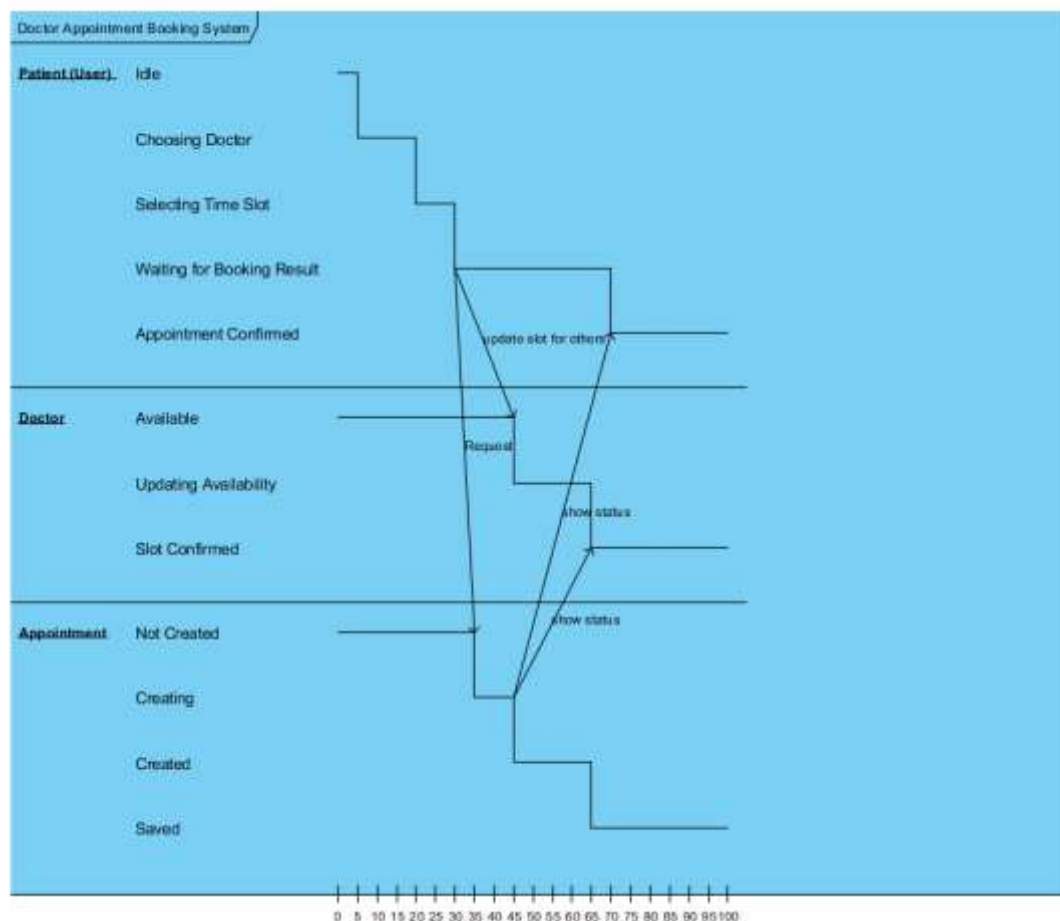
The Communication Diagram illustrates the message flow and interactions between objects during the appointment booking process. It shows how the **Patient**, **Doctor**, and **Appointment** objects communicate and exchange messages in order to complete the booking.

This diagram focuses on the structure of the interaction and the order in which messages are passed, helping to better understand system behavior during runtime.



3.4 Timing Diagram

The Timing Diagram explains how the states of different objects change over time during the appointment booking process. It shows the timeline of events from when the patient selects a doctor to when the appointment is confirmed.



4. Implementation

- *GitHub* Repository

All source code, UML files, and project structure are available in the following GitHub repository:

<https://github.com/git-rafaykhan/doctor-oppointment-booking.git>

The repository includes:

- Complete source code (backend/frontend and UML Designs)
- Exception handling
- UML and design files

5. Test Cases

I. Test Case 1 – Valid Login Credentials

Test Case Summary and Documentation

Use Case ID	TC_LOGIN_001
Test Case Description / Partition Tested:	Valid Login Credentials
Created By:	TBD (Dev/Tester)
Reviewed By:	TBD
Version:	1.0 (TDD Design)

Prerequisites and Test Data

S#	Prerequisites	S#	Test Data / Input
1	System is online and accessible	1	Email: user@example.com
2		2	Password: Test@123

Test Scenario and Steps

Test Scenario: Verify that the system allows a user to log in successfully using valid credentials.

Step#	Step Details	Expected Results	Actual Results	Pass/Fail
1	Navigate to Login Page	Login page should load	Login page appear	Pass

Step#	Step Details	Expected Results	Actual Results	Pass/Fail
2	Enter valid Email & Password	Credentials should be accepted	Crediantials accepted	Pass
3	Click Login Button	Redirected to user dashboard	Redirected	Pass

Test Case 1 – Invalid Login Credentials

Test Case Summary and Documentation

Use Case ID	TC_LOGIN_001
Test Case Description / Partition Tested:	Invalid Login Credentials
Created By:	TBD (Dev/Tester)
Reviewed By:	TBD
Version:	1.0 (TDD Design)

Prerequisites and Test Data

S#	Prerequisites	S#	Test Data / Input
1	System is online and accessible	1	Email: wrong@example.com
		2	Password: wrongpass123

Test Scenario and Steps

Test Scenario: Verify that the system allows a user to log in successfully using valid credentials.

Step#	Step Details	Expected Results	Actual Results	Pass/Fail
1	Navigate to Login Page	Page loads successfully	Login page appear	Pass
2	Enter valid Email & Password	Credentials should be accepted	Crediantials accepted	Pass
3	Click Login Button	Error message: <i>"Invalid Credentials"</i>	Invalid Crediantials	Pass

II. Test Case 2 – Admin Adds a New Doctor to the Platform

Test Case Summary and Documentation

Use Case ID	TC_ADMIN_001
Test Case Description / Partition Tested:	Add New Doctor (Admin Functionality)
Created By:	TBD (Dev/Tester)
Reviewed By:	TBD
Version:	1.0 (TDD Design)

Prerequisites and Test Data

S#	Prerequisites	S#	Test Data / Input
1	Admin is logged into Admin Dashboard	1	Doctor Name: Dr. Abdul Rafay
2	Admin has access to <i>Manage Doctors</i> module	2	Specialization: Cardiologist
		3	Email: rafay@Mediconnect.com
		4	Phone: 0300-1234567
		5	Fee: 1500 PKR

Test Scenario and Steps

Test Scenario: Verify that the system allows a user to log in successfully using valid credentials.

Step#	Step Details	Expected Results	Actual Results	Pass/Fail
1	Navigate to Admin Dashboard → Manage Doctors	Manage Doctors page should load	Login page appear	Pass
2	Click Add New Doctor	Add Doctor form should appear	Crediantials accepted	Pass
3	Enter Doctor Name, Specialization, Email, Phone, and Fee etc	Form accepts valid inputs	Invalid Crediantials	Pass

Step#	Step Details	Expected Results	Actual Results	Pass/Fail
4	Click Save / Add Doctor	Doctor should be added successfully	Doctor Added successfully	Pass

Test Case 2 – Admin Adds a New Doctor to the Platform (Invalid Case)

Test Case Summary and Documentation

Use Case ID	TC_ADMIN_001
Test Case Description / Partition Tested:	Add Doctor – Invalid/Incomplete Details
Created By:	TBD (Dev/Tester)
Reviewed By:	TBD
Version:	1.0 (TDD Design)

Prerequisites and Test Data

S#	Prerequisites	S#	Test Data / Input
1	Admin is logged into Admin Dashboard	1	Doctor Name: <i>(empty)</i>
2	Admin has access to <i>Manage Doctors</i> module	2	Specialization: <i>(empty)</i>
		3	Email: invalid-email-format
		4	Phone: 123 (invalid number)
		5	Fee: -500 (invalid fee)

Test Scenario and Steps

Test Scenario: Verify that the system allows a user to log in successfully using valid credentials.

Step#	Step Details	Expected Results	Actual Results	Pass/Fail
1	Navigate to Admin Dashboard → Manage Doctors	Manage Doctors page should load	Login page appear	Pass

Step#	Step Details	Expected Results	Actual Results	Pass/Fail
2	Click Add New Doctor	Add Doctor form should appear	Form Appeared	Pass
3	Leave required fields blank or enter invalid values (invalid email, short phone number, negative fee)	System should highlight invalid fields	Invalid Crediantials	Pass
4	Click Save / Add Doctor	System should NOT save the doctor	System should no add Doctor	Pass